Lab 2 – Assembly Programming, Signal Measurement, & Peripheral Interfacing on the mbed
Part 1: PWM & Timer Interrupt

Embedded Processors UESTC2004 & UESTCHN2004 School of Engineering, University of Glasgow

Student Name: Li Chenyu

Student Number: 3073568

UESTC 2004 Lab Assessment Criteria

Generally, each individual lab will be evaluated using the below criteria.

- Demonstrating a functional embedded system

  This means each student will need to demonstrate a functional system for each task in the lab manual to the GTA and member of staff to get the full mark.

- Lab Questions

- Reflection and significance

  After the completion of programming and testing each of the “Tasks”, you will be required to reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves “What you did?”, “What it means?”, and “What next?”, meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

- Adherence to coding good practice
  - Meaningful variable names – please avoid names like “x”, “y”, or personal names
  - Using block comments to explain what your code does at the beginning and using line comments to clarify what this line of code/ does.

- Consistent indentation – Indentation helps in making the code readable and easy to interpret. See an example of what to do and what to avoid, below:

Good practice ✓

```
#include “mbed.h”
char switch_word ;                    //the word we will send char
recd_val;                             //the received value
int main() {
    async_port.baud(9600);                    //set baud rate to 9600
        recd_val = 0;
        while (1)
        {
            switch_word=0xa0;
        }
}
```

Avoid this ✗

```
#include “mbed.h”
char switch_word ;
char recd_val;
int main() {
async_port.baud(9600);
recd_val = 0;
while (1)
{
switch_word=0xa0;
}
}
```

All the best of luck.

The Embedded Processors Team,

Lab Objective

- Write and execute assembly language programs to perform arithmetic operations using an ARM simulator, reinforcing low-level programming concepts.
- Understand register operations and instruction sets to see how they impact program execution in an ARM-based environment.
- Analyse embedded output signals using an oscilloscope to deduce the functionality of an unknown executable, including identifying PWM characteristics and digital timing behaviour.

- Demonstrate practical use of hardware timers and interrupts (using Ticker) to implement non-blocking periodic tasks in real-time embedded systems.
- Design and implement PWM-based control logic by configuring duty cycles and frequency on a microcontroller, and validate the output using measurement tools.

Requirements

Software:

- Keil arm online compiler (https://studio.keil.arm.com/auth/login/) – You will need to create an account to use the online compiler.
- Download and install the ARMSim#1.91 from Moodle-> Lab Material -> "Lab 2"(Folder).

  Note: To run ARMSim# 1.91 you must be running a **Windows system** with a **.NET** Framework. To run it on Linux or Mac OS, follow the instructions on its official website. **(Please do this before attending the lab)**

Task 1: Programming in Assembly

Task 1.1: ARMSim# is not an editor, so you will need to create and edit your ARM assembly program with a separate text editor (such as Notepad++). Open your favourite text editor and type in the following simple program:

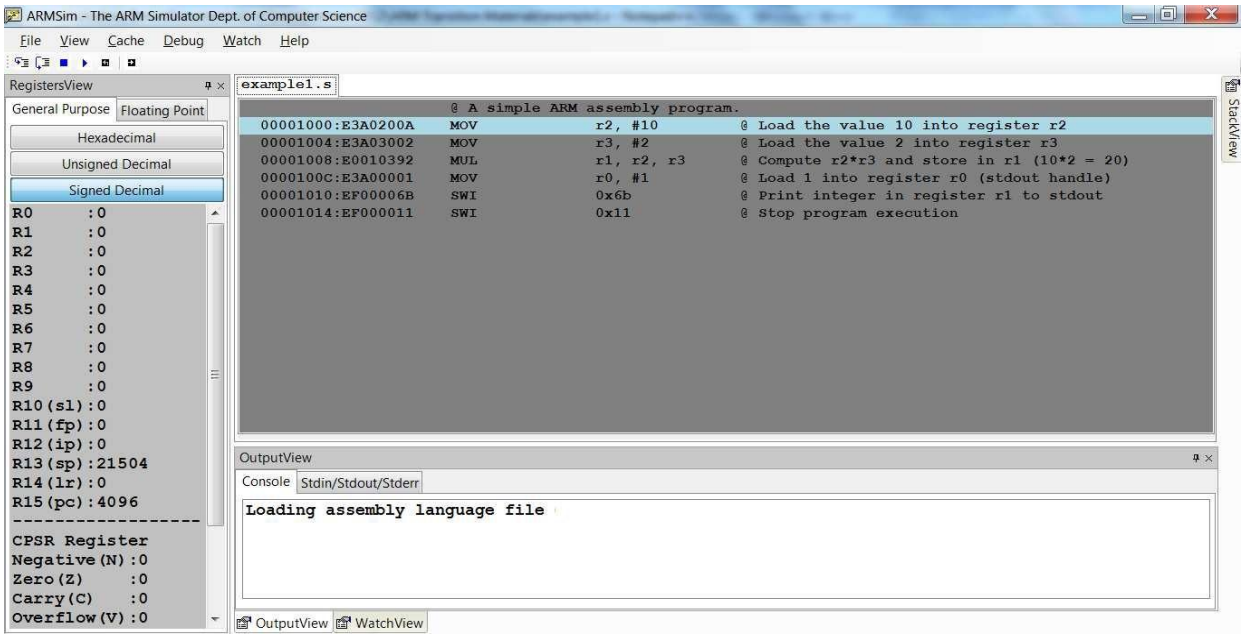
Do not worry if some of these instructions are unfamiliar at the moment, they will be covered in time. Save your program as example1.s – note the '.s' file extension is required in order for ARMSim# to read your file. Once you have saved your ARM program, you need to load it into ARMSim#. Open the ARMSim# program, select 'File -> Load' from the menu, and select your example1.s file. It should open your program and make sure it is without syntactic errors.
Your screen will look like the one below. Your example1.s file should look like below:

```
@ A simple ARM assembly program.
MOV r2, #10 @ Load value 10 into register
r2
MOV r3, #2 @ Load value 2 into register r3
MUL r1, r2, r3 @ Compute r2*r3 & store in r1(10*2 =
20)
MOV r0, #1 @ Load 1 into register r0 (stdout handle)
SWI 0x6b @ Print integer in register r1 to stdout
SWI 0x11 @ Stop program execution
```

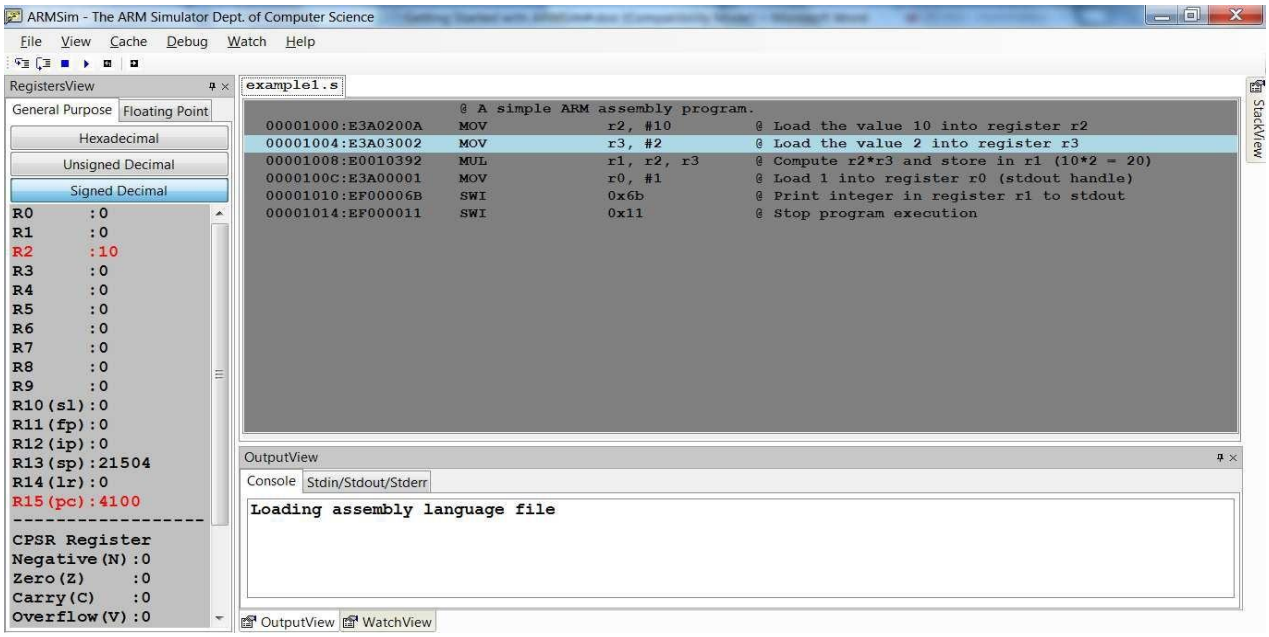
On the left side of the screen, the registers for the simulated ARM processor are displayed. The source code window is in the centre, while the console window is at the bottom.

There are three options for running (simulating) your assembly language program – run,

step into, and step over. The two that will likely be of most use (and their toolbar icons) are (i) run( ) and (ii) step into ( ). Run (keyboard shortcut F5) will run your assembly language program from start to finish, while step into (keyboard shortcut F11) will execute one instruction at a time, allowing you to see how each instruction is modifying the state.

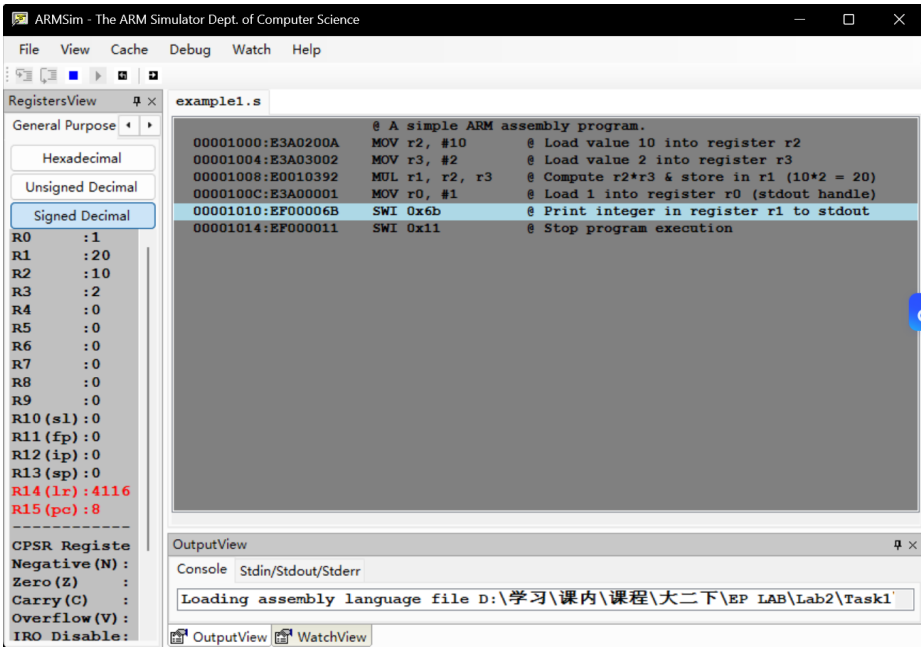


Press “Step Into”, and you should see a screen like the one below.



On the left hand side, you will see that the registers that have been changed by the executed instruction are red. In the source code window, you will notice that the highlighted blue line is now the next instruction – is the instruction pointed to by the program counter,

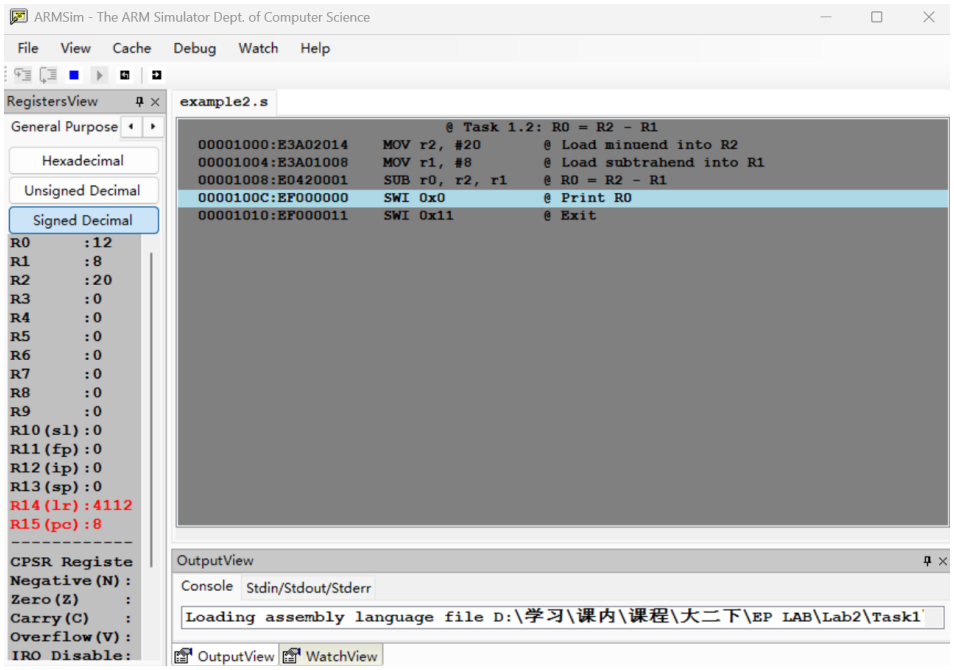
and will be executed next. Try stepping through the remainder of the simple program, verifying that the registers are changing as expected.



The program executed exactly as expected. The instruction MOV r2, #10 loaded 10 into R2, MOV r3, #2 loaded 2 into R3, and MUL r1, r2, r3 computed their product and stored 20 in R1. The instruction MOV r0, #1 set R0 as the stdout handle, after which SWI 0x6b printed the value 20 to the console, and SWI 0x11 terminated the program. All register updates were correct and the expected output was produced without any errors.

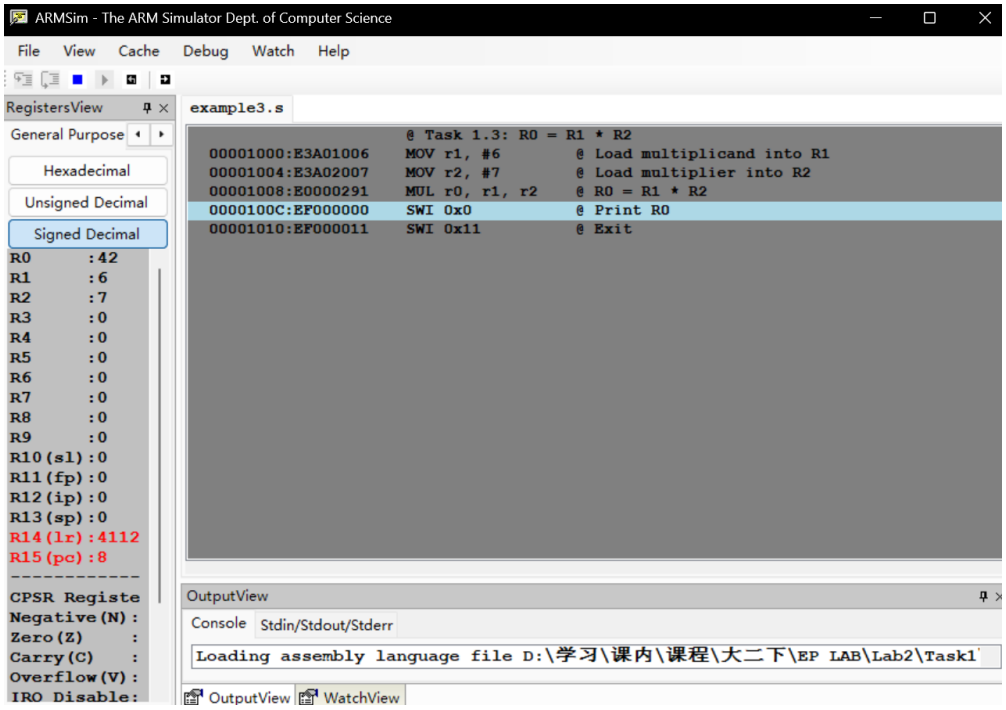
Task 1.2: Complete the following tasks with reference to the codes in Task 1.1

Write a program to perform the subtraction of a value in register 1 from a value in register 2 and the result should be stored in register 0. Once completed, show the demo to the GTA to get your mark.



The SUB instruction computes the difference between two registers. Here, R2 holds the minuend (20) and R1 holds the subtrahend (8). After execution, R0 stores the result (12).

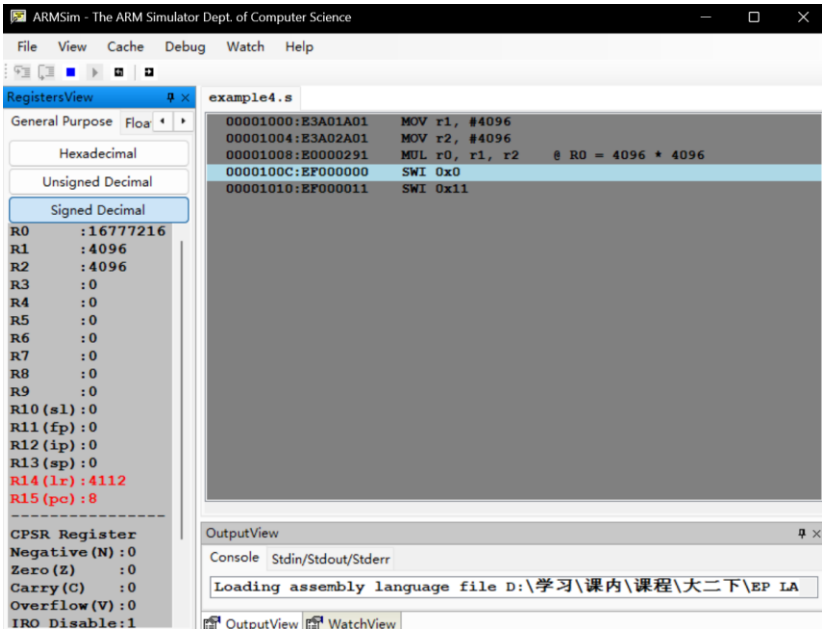
Task 1.3: Write a program to multiply the value in register 1 with the value in register 2 and store the result in register 0.



The MUL instruction multiplies two registers and stores the product in a destination register. The syntax is MUL dest, src1, src2. Here, R1 (6) and R2 (7) are multiplied, and the result

(42) is placed in R0.

Task 1.4: With the multiplication program, set registers 1 and 2 to 4096. What is the result from the simulator? **[Answer in the below space]**



4096 × 4096 = 16,777,216 (Hexadecimal: 0x01000000). The product 16,777,216 fits comfortably within a 32-bit register, whose maximum value is 4,294,967,295. Therefore, no overflow occurs, and the MUL instruction stores the exact 32-bit result in R0. The simulator displays 16777216 in the console or R0 register view.

Task 1.5: Debugging

The following are some debugging hints that you might consider if Task 1 output is not as expected:

- **Syntax Errors in Assembly:** Ensure proper instruction format and case sensitivity.
- **Incorrect Register Values:** Step through code using the "Step Into" function.
- **Unexpected Output:** Double-check operand order and register assignments.

Task 2: PWM & Interrupts on mbed L432KC

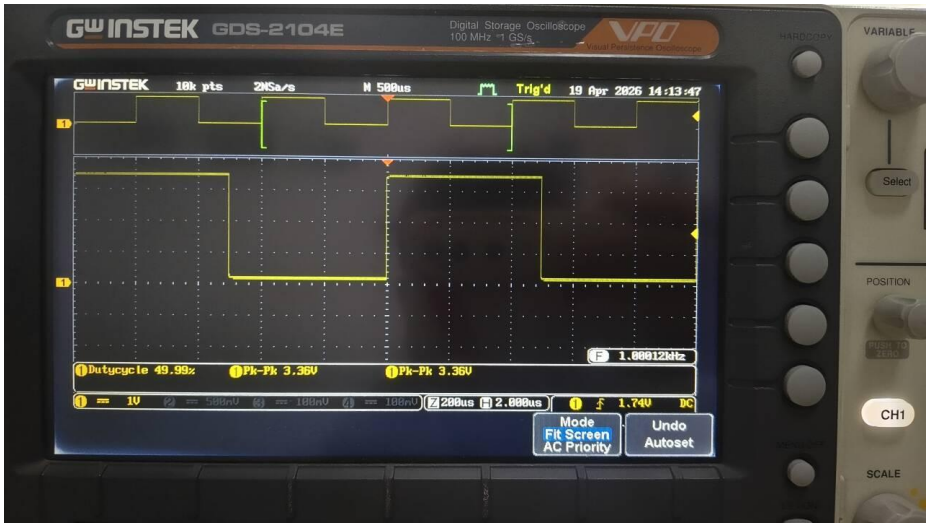
Context/Scenario:
You are an embedded systems engineer developing part of a control unit for an environmental monitoring device. The device uses a microcontroller to process analogue signals from sensors, and control output actuators via PWM. As part of prototyping, you need to analyse the behaviour of a compiled program running on the mbed L432KC board. You are not given the code, only the executable file which you can find on Moodle in the "Lab 2" folder called "***UESTC_UESTCHN2004_Lab2.bin***".

You are tasked with reverse-engineering the functionality of the binary and extending the functionality based on your analysis. What you will need:

1. L432KC mbed board
2. Jumper wires
3. Oscilloscope (Two Channels). Hint: When using the oscilloscope to measure an unknown signal consider switching the trigger mode to Normal as low-frequency signals may not be picked up by the oscilloscope's "AUTO" capture.

Task 2.1: Understanding the Digital Output

- Using an oscilloscope, measure the signal on pin D5. Paste a picture of the oscilloscope screen below.



  - What type of waveform do you observe? **[Answer below]**

    Perfect standard square wave, 1.00012 kHz, 49.99% duty cycle, 3.36 V peak-to-peak, extremely sharp edges, no overshoot or noise.

  - What is the period and frequency of the signal? **[Answer below]**

    The period is 1 ms and the frequency is 1.0 kHz.

  - Based on your measurements, how often does the signal toggle? **[Answer**

below]

The signal toggles for every 0.5ms.

- Assume the signal on D5 is toggled in software. Estimate the time delay or loop timing used in the code. What instruction or function might be responsible for this?

  The delay between successive toggles is approximately 0.5ms. In mbed C++, this is typically implemented using the blocking function wait() or ThisThread::sleep_for(). If written in assembly, it would be a busy-wait loop using a decrement counter and branch instruction such as SUBS R0, #1 / BNE loop.

Task 2.2: Investigating the PWM Signal

- Connect an oscilloscope to **D6**, where a PWM signal is being generated.
  - Paste a picture of the output below.

  - What is the **period** of the PWM signal? **[Answer below]**

    The period is 2.000s.

  - From your measurements, calculate the **PWM frequency** in Hz. **[Answer below and show your workings]**

    The frequency = 1 / period = 0.5Hz.

- Based on your findings:
  - What do you think the **PWM pulse width value** is (in seconds)? **[Answer below and show your workings]**

    The measured duty cycle is 25%, combined with full period of 2s. The PWM pulse width value is 2/4s = 0.5s.

  - What percentage of the time is the signal HIGH (duty cycle)? **[Answer below and show your workings]**

The measured duty cycle is 25%.

- Based on your answers above, write the lines of code used to generate the PWM signal. **[Answer below]**

```
#include "mbed.h"

DigitalOut pin(D10);                // Digital output on pin D5

int main() {
    while (1) {
        pin = 1;                    // High for 0.5s (25% of 2s)
        ThisThread::sleep_for(500ms);

        pin = 0;                    // Low for 1.5s
        ThisThread::sleep_for(1500ms);
    }
```

Task 2.3: Code Function Deduction

- From your observations, describe what the program is likely doing in high-level terms. Consider:
  - What kind of output is being generated on D5 and why? **[Answer below]**

    D5 generates a 1 kHz square wave with a 50% duty cycle. This serves as a heartbeat or status indicator, providing visual confirmation that the main program loop is executing correctly and the microcontroller is not stuck or crashed.

  - What is the purpose of toggling the digital pin in this context? **[Answer below]**

    Toggling D5 demonstrates precise timing control and provides a simple debugging mechanism. The 1 kHz frequency indicates that the system clock and delay functions are configured correctly. In embedded systems, such a signal is often used to verify that the firmware is alive and running at the expected speed.

Task 2.4: Extension Task – Using a Ticker Interrupt

- Having inferred what the code does, extend it by adding a Ticker to Toggle Between Two Duty Cycles (30% and 70%). The PWM frequency should remain the same as Tasks 2.1 to 2.3.

[Paste your code below]

```
#include "mbed.h"

// PWM output pin
PwmOut pwmPin(D10);

// Heartbeat LED pin
DigitalOut heartbeatPin(D5);

// Periodic interrupt object
Ticker dutyCycleTicker;

// Duty cycle state flag
volatile bool highDutyCycle = false;

// PWM parameter definitions
#define PWM_PERIOD_SECONDS     1.0f        // 2 second period
#define DUTY_CYCLE_LOW         0.30f       // Low duty cycle 30%
#define DUTY_CYCLE_HIGH        0.70f       // High duty cycle 70%
#define TOGGLE_INTERVAL_SEC    5.0f        // Toggle interval 5 seconds

// Ticker ISR: Toggle duty cycle flag
void toggleDutyCycleISR() {
    highDutyCycle = !highDutyCycle;
}

// Main function
int main() {
    // Configure PWM period
    pwmPin.period(PWM_PERIOD_SECONDS);

    // Initialize with low duty cycle
    pwmPin.write(DUTY_CYCLE_LOW);
    highDutyCycle = false;

    // Attach Ticker interrupt, triggers every 5 seconds
    dutyCycleTicker.attach(&toggleDutyCycleISR, TOGGLE_INTERVAL_SEC);

    // Main loop
    while (1) {
```

```
        // Output 1 kHz heartbeat on D5 (50% duty cycle)
        heartbeatPin = 1;
        wait_us(500);
        heartbeatPin = 0;
        wait_us(500);

        // Update PWM duty cycle based on flag
        if (highDutyCycle) {
            pwmPin.write(DUTY_CYCLE_HIGH);
        } else {
            pwmPin.write(DUTY_CYCLE_LOW);
        }
    }
}
```

- Validate your code by measuring the period, frequency, and duty cycle of the generated PWM signals using an oscilloscope. Paste pictures of your measurement and detail your answer to show how you measured/calculated each.

[Answer below]

Directly read physical quantities such as the period and frequency of the signal through the Measure mode of the oscilloscope. For two different PWM duty cycles, press the Pause button of the oscilloscope at different time periods during the experiment to capture stable waveforms with a 30% low duty cycle and a 70% high duty cycle respectively. Then read the corresponding duty cycle values from the measurement interface. It is also verified that parameters including the period of the PWM signal and the frequency and period of the heartbeat signal are all consistent with the design expectations.

Task 4: Reflection

After the completion of programming and testing of the tasks please reflect, in a few lines, on your understanding of the tasks and their significance. This is a chance for you to ask yourselves **"What you did?", "What it means?", "What next?"**, and/or How else, more efficiently, can this be done?, meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. **Please, do not provide a summary of what you did as it will not be accepted.**

[Answer below]

In Task 1, I wrote and executed ARM assembly programs using ARMSim#, implementing subtraction and multiplication operations at the register level. This required understanding the syntax of data processing instructions such as MOV, SUB, and MUL, as well as the role of software interrupts for I/O and program termination. In Task 2, I reverse-engineered an unknown binary file running on the mbed L432KC. Using only an oscilloscope, I characterised a 1 kHz digital heartbeat on D5 and a slow 0.5 Hz PWM signal with 25% duty cycle on D6. Finally, I extended the system by writing a C++ program that uses a Ticker interrupt to dynamically switch the PWM duty cycle between 30% and 70% without blocking the main execution thread.

Task 1 showed the relationship between high-level code and processor execution. Seeing how arithmetic operations translate to individual register manipulations reinforced the concept that all software ultimately runs as sequences of simple hardware instructions. Task 2 simulated a real-world embedded engineering scenario: diagnosing a black-box system using only hardware tools. Understanding PWM is foundational because it enables efficient analog control of actuators using purely digital outputs. The shift from blocking delays to interrupt-driven design illustrates a critical principle in real-time systems: the CPU should never be idle waiting when it could be servicing other tasks. The Ticker approach offloads timing to hardware, freeing the main loop for sensor polling, communication, or state machine logic.

The skills developed in this lab are directly transferable to more complex embedded applications. PWM can be combined with feedback from encoders or ADCs to implement closed-loop motor speed or position control. The Ticker pattern demonstrated here forms the basis of task scheduling in bare-metal systems and is a stepping stone to understanding Real-Time Operating Systems. A more efficient implementation of the D5 heartbeat would use a second PWM channel configured for 50% duty cycle at 1 kHz, eliminating all software overhead. Additionally, the Ticker interval could be made adjustable via a potentiometer input, creating an interactive system where the user controls the modulation rate. These extensions bridge the gap between simple peripheral control and fully autonomous embedded devices.

Appendix

Task 1.1

```
@ A simple ARM assembly program.
00001000: E3A0200A    MOV r2, #10       @ Load value 10 into register r2
00001004: E3A03002    MOV r3, #2        @ Load value 2 into register r3
00001008: E0010392    MUL r1, r2, r3    @ Compute r2*r3 & store in r1 (10*2 = 20)
0000100C: E3A00001    MOV r0, #1        @ Load 1 into register r0 (stdout handle)
00001010: EF00006B    SWI 0x6b          @ Print integer in register r1 to stdout
00001014: EF000011    SWI 0x11          @ Stop program execution
```

Task 1.2

```
@ Task 1.2: R0 = R2 - R1
00001000: E3A02014    MOV r2, #20       @ Load minuend into R2
00001004: E3A01008    MOV r1, #8        @ Load subtrahend into R1
00001008: E0422001    SUB r0, r2, r1    @ R0 = R2 - R1
0000100C: EF000000    SWI 0x0           @ Print R0
00001010: EF000011    SWI 0x11          @ Exit
```

Task 1.3

```
@ Task 1.3: R0 = R1 * R2
00001000: E3A01006    MOV r1, #6        @ Load multiplicand into R1
00001004: E3A02007    MOV r2, #7        @ Load multiplier into R2
00001008: E0000291    MUL r0, r1, r2    @ R0 = R1 * R2
0000100C: EF000000    SWI 0x0           @ Print R0
00001010: EF000011    SWI 0x11          @ Exit
```

Task 1.4

```
00001000: E3A01A01    MOV r1, #4096
00001004: E3A02A01    MOV r2, #4096
00001008: E0000291    MUL r0, r1, r2    @ R0 = 4096 * 4096
0000100C: EF000000    SWI 0x0
00001010: EF000011    SWI 0x11
```

Task 2.2

```
#include "mbed.h"

DigitalOut pin(D10);              // Digital output on pin D5

int main() {
   while (1) {
      pin = 1;                    // High for 0.5s (25% of 2s)
      ThisThread::sleep_for(500ms);

      pin = 0;                    // Low for 1.5s
      ThisThread::sleep_for(1500ms);
   }
```

Task 2.4

```
   #include "mbed.h"

   // PWM output pin
```

```
PwmOut pwmPin(D10);

// Heartbeat LED pin
DigitalOut heartbeatPin(D5);

// Periodic interrupt object
Ticker dutyCycleTicker;

// Duty cycle state flag
volatile bool highDutyCycle = false;

// PWM parameter definitions
#define PWM_PERIOD_SECONDS     1.0f       // 2 second period
#define DUTY_CYCLE_LOW         0.30f      // Low duty cycle 30%
#define DUTY_CYCLE_HIGH        0.70f      // High duty cycle 70%
#define TOGGLE_INTERVAL_SEC    5.0f       // Toggle interval 5 seconds

// Ticker ISR: Toggle duty cycle flag
void toggleDutyCycleISR() {
   highDutyCycle = !highDutyCycle;
}

// Main function
int main() {
   // Configure PWM period
   pwmPin.period(PWM_PERIOD_SECONDS);

   // Initialize with low duty cycle
   pwmPin.write(DUTY_CYCLE_LOW);
   highDutyCycle = false;

   // Attach Ticker interrupt, triggers every 5 seconds
   dutyCycleTicker.attach(&toggleDutyCycleISR, TOGGLE_INTERVAL_SEC);

   // Main loop
   while (1) {
      // Output 1 kHz heartbeat on D5 (50% duty cycle)
      heartbeatPin = 1;
      wait_us(500);
      heartbeatPin = 0;
      wait_us(500);

      // Update PWM duty cycle based on flag
      if (highDutyCycle) {
         pwmPin.write(DUTY_CYCLE_HIGH);
      } else {
         pwmPin.write(DUTY_CYCLE_LOW);
      }
   }
}
```

Lab Score

Marking Criteria	Student's Score
Task 1.2 (5%) Task 1.3 (5%) Task 1.4 (5%)	
Task 2.1 (10%) Task 2.2 (10%) Task 2.3 (10%) Task 2.4 (35%)	
Task 4 Reflection (5%)	
Adherence to coding good practice (15%)	